SRGAN Image Super Resolution and Binary Classification

Detailed midterm implementation and reproducibility report

Owen Klimek | University of North Carolina at Charlotte | 27 October 2026

1 Purpose and main findings

This experiment studied whether images reconstructed from low-resolution inputs could support a cats-versus-dogs classifier. Classifier A learned from real 128 × 128 images. A compact super-resolution generative adversarial network (SRGAN) learned to reconstruct 128 × 128 RGB images from 32 × 32 inputs. Classifier B then learned from the generated versions of the same fitting images, using the same MobileNetV2 initialization and training schedule as A.

The completed run used 24,930 images, 5 generator warm-up epochs and 150 full adversarial epochs. On the same real test images, A achieved 98.0612% accuracy and B achieved 97.1253%. On the same SRGAN test inputs, B achieved 94.3442% accuracy and A achieved 94.0500%. Thus, generated-image training did not improve the primary real-image test result, but B performed better under the two reconstructed input conditions. Bicubic interpolation retained slightly higher test PSNR and SSIM than SRGAN.

All reported classifier metrics were independently recomputed from the saved per-image prediction files. Dataset hashes, split membership, label alignment and the 155 GAN epoch records were checked against the exported experiment records [1].

Assignment requirements and evidence

Requirement	Implementation and report location
Classifier A with transfer learning	MobileNetV2 with 128 × 128 input; Section 4
Super resolution from 32 to 128 pixels	Paired LR and HR images, residual generator and discriminator; Sections 3 and 5
Scaled and transformed examples	Recorded notebook figures reproduced in Section 3
Classifier B trained on generated images	Generated fitting images only, inherited labels; Section 4
At least 150 SRGAN epochs	150 adversarial epochs plus 5 warm-up epochs; Sections 6 and 7
70 percent training and 30 percent testing	Stratified outer split with an inner validation set; Section 2
Accuracy F1 and AUC comparison	Six matched-input evaluations and worked calculations; Sections 9 and 10
Periodic saving for Colab sessions	Full-state checkpoints after every completed epoch; Section 12
Reproduction document in GitHub	Step-by-step procedure and repository contents; Sections 13 and 15

2 Dataset and separation of training and testing

Dataset identity and labels

The midterm reused the prepared cats-versus-dogs image collection from Assignment 1, supplied as dogcat_dataset.zip. The saved manifest identifies files under dogs-vs-cats-classification, including its former train, validation and test folders. These old folders were treated as source-image locations and were pooled before the new midterm split. No Assignment 1 classifier weights were reused; the new classifiers started from ImageNet initialization.

Class	Numeric label	Images
Cat	0	12,472
Dog	1	12,458
Total	—	24,930

Pillow opened each candidate image, corrected EXIF orientation, converted it to three-channel RGB, and resized it to 128 × 128 with bicubic interpolation. Images were stored as uint8 arrays. The midterm records report zero unreadable or unknown-label exclusions and zero exact duplicate removals. These counts describe this pass through the already prepared Assignment 1 collection; they do not describe any earlier cleaning of the original source download.

Stratified outer split and inner validation

A stratified split with seed 42 assigned 70% of image IDs to the training pool and 30% to the final test set. Ten percent of the training pool was then reserved for validation. The effective full-dataset allocation was approximately 63% fitting, 7% validation and 30% testing. This maintains the required outer 70/30 split while keeping a separate set for model selection.

Subset	Cats	Dogs	Total	Share of all images
Fitting	7,857	7,848	15,705	62.996%
Validation	873	873	1,746	7.004%
Training pool	8,730	8,721	17,451	70.000%
Final test	3,742	3,737	7,479	30.000%

The training-pool row is the sum of fitting and validation, not an additional disjoint subset. G, A and B fitted only the same 15,705 fitting IDs. Validation targets monitored reconstruction and selected classifier weights. The 7,479 test IDs supplied final evaluation and were never passed to a weight-update step.

Checks against leakage and accidental data changes

Exact SHA-256 hashes of the prepared RGB pixels were checked before splitting. The saved image_manifest.json contains index, relative source path, label and pixel hash. split_indices.json preserves exact membership. Verification confirmed that the three disjoint subsets cover every image once, all 24,930 recorded pixel hashes are unique, and every prediction file uses the saved test IDs and matching labels. Exact-pixel checks do not exclude near-duplicate views or repeated photographs of the same animal; those remain limitations of an image-level split.

3 Resizing normalization and image transformations

Creating a paired super resolution problem

Each real RGB target has shape 128 × 128 × 3. Its low-resolution input is produced by antialiased bicubic downsampling to 32 × 32 × 3, followed by clipping to [0,1]. The scale factor is four in each spatial direction and sixteen in total pixel count. Bicubic enlargement of the LR image supplies a deterministic baseline; it does not learn to recover class-specific detail.

$$Spatial\ \ scale\ = \frac{128}{32} = 4\,; \quad pixel\text{-}count\ \ ratio\ = 4^2\ = 16$$



Figure 1. Recorded example of the real 128 × 128 target, its 32 × 32 input and the bicubic 128 × 128 baseline.

Normalization for each network

Prepared images use uint8 values from 0 to 255. Dataset batches are converted to float32 and divided by 255, giving [0,1] inputs and GAN targets. The generator uses a sigmoid RGB output in the same range. Classifiers contain their MobileNetV2 rescaling internally: 2x − 1, giving [-1,1] for unaugmented inputs [3]. The frozen VGG19 network receives RGB values multiplied by 255 and its own ImageNet preprocessing, which changes RGB to BGR and subtracts the pretrained channel means [6].

$$x\ = \frac{I}{255}\,; x_{MobileNetV2} = 2x\ -\ 1$$

Training transformations and pair alignment

Classifier augmentation applies a random horizontal flip, a rotation factor of 0.04 (up to approximately ±14.4 degrees), a zoom factor of 0.10, and a contrast factor of 0.10. Rotation and zoom use reflected boundaries. Both classifiers use this same policy. The preview clips transformed images to [0,1] for display. Training augmentation is performed inside the classifier; contrast adjustment can alter the pre-normalization intensity range. Validation and test inputs receive no random transformations.



Figure 2. Six recorded training transformations of one fitting image. Variations demonstrate pose, framing and contrast changes; they are not six independent labeled subjects.

GAN training uses a shared horizontal flip applied to HR before LR creation. Downsampling the already flipped target keeps LR and HR spatially aligned. Applying independent flips to the two images would give contradictory supervision. LR generation and all validation/test reconstruction inputs are otherwise deterministic.

4 Transfer learning for classifiers A and B

Architecture and controlled initialization

Both classifiers use ImageNet-pretrained MobileNetV2 without its original classification top. A 128 × 128 RGB input passes through training augmentation, the internal MobileNetV2 rescaling, the pretrained feature extractor, global average pooling, dropout of 0.30, and one sigmoid neuron. The output is the estimated probability of dog. Binary cross-entropy supplies the training loss. ImageNet features provide a useful starting representation of edges, textures and shapes; the small task-specific head learns the new binary decision.

classifier_initial.keras was saved before fitting A. A and B each load that same initialization. B is not initialized with A's learned weights. The architecture, fitting IDs, batch size, epoch budgets, augmentation policy and classifier selection criterion are held constant. The principal training-data difference is that A sees real targets and B sees generated reconstructions.

Stage	Epochs for each model	Learning rate	Trainable parameters
Head training	5	0.001	New classifier head; backbone frozen
Fine tuning	3	0.00001	Last 30 backbone layers eligible; batch normalization frozen

The backbone is called with training=False, preserving inference-mode batch-normalization behavior. During fine-tuning, the final 30 layers can train except batch-normalization layers; lower layers remain frozen. The lower learning rate limits abrupt changes to pretrained features. Each stage runs its full configured epoch budget. The implementation selects weights by the lowest validation loss, rather than stopping GAN training early or selecting on test accuracy.

Generating the training data for B

After adversarial epoch 150, G is used in inference mode to reconstruct every source image from its 32 × 32 LR version. Outputs are clipped, multiplied by 255, rounded and stored as uint8. Source labels and split IDs are retained. Only the 15,705 generated fitting images train B, while the 1,746 generated validation images select its weights. Generated test images are kept for evaluation. Storing the generated images once avoids rerunning G in every B training epoch.

Selected classifier	Best head validation loss	Best fine-tuning validation loss	Exported stage
A	0.060253	0.044653	Fine tuning epoch 3
B	0.131565	0.113975	Fine tuning epoch 3

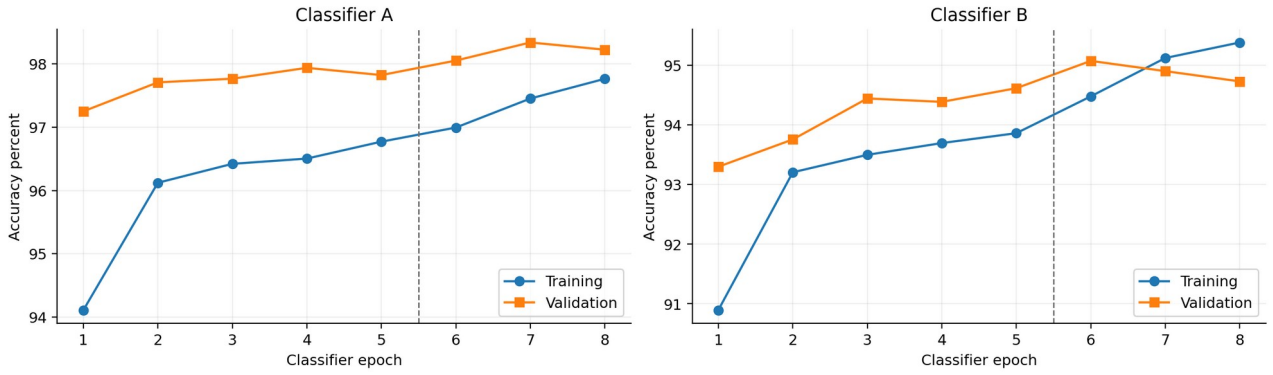


Figure 3. Accuracy derived from saved classifier logs. Epochs 1–5 train the head; epochs 6–8 fine-tune. A validates on real images and B on SR images, so their validation curves describe different input domains.

5 SRGAN architecture and image generation

Generator

The generator is a compact adaptation of the SRGAN approach [2]. It accepts a 32 × 32 RGB image and predicts a 128 × 128 RGB reconstruction. Most convolutions operate at low resolution to limit computation. Residual additions provide short paths through the network, while a long skip adds the early feature representation after the residual stack. PReLU provides a learnable negative activation slope.

Generator component	Operation	Output shape
Input	Three RGB channels	32 × 32 × 3
Initial features	9 × 9 convolution, 32 filters, PReLU	32 × 32 × 32
Residual stack	8 blocks; each has two 3 × 3 convolutions, BN and a local addition	32 × 32 × 32
Long skip	3 × 3 convolution, BN, addition with early features	32 × 32 × 32
First enlargement	3 × 3 convolution to 128 channels, pixel shuffle by 2, PReLU	64 × 64 × 32
Second enlargement	Same enlargement block	128 × 128 × 32
RGB prediction	9 × 9 convolution, 3 filters, sigmoid, float32 output	128 × 128 × 3

Pixel shuffle rearranges groups of feature channels into spatial positions. A 32 × 32 × 128 feature tensor becomes 64 × 64 × 32 without inventing additional tensor elements. Two such enlargements produce the required fourfold spatial scale. PixelShuffle is registered as a serializable Keras layer so the exported generator can be loaded with its class definition available.

Discriminator

Convolution	Filters	Stride	Spatial output
1	32	1	128 × 128
2	32	2	64 × 64
3	64	1	64 × 64
4	64	2	32 × 32
5	128	1	32 × 32
6	128	2	16 × 16
7	256	1	16 × 16
8	256	2	8 × 8

All discriminator convolutions use 3 × 3 kernels and LeakyReLU with slope 0.2. Batch normalization follows every convolution except the first, with momentum 0.8. Global average pooling is followed by a dense layer of 256 units, LeakyReLU, and a single float32 logit. Binary cross-entropy uses from_logits=True. This discriminator distinguishes real from generated images; it does not predict cat versus dog.

Frozen perceptual feature network

ImageNet VGG19 features from block3_conv4 provide the content representation. Its weights remain frozen. The earlier VGG feature layer, eight 32-channel generator blocks, global-pooling discriminator, sigmoid output and added L1 term are explicit implementation choices. Results should be described as this compact SRGAN adaptation, not an exact reproduction of every architecture and loss setting in the original paper.

6 Loss functions and alternating training

Reconstruction warm up

The generator first trains for five complete epochs using mean absolute pixel error. This stage teaches the LR-to-HR mapping before the adversarial objective is introduced. Warm-up does not count toward the required 150 adversarial epochs. Let z denote LR input, x the corresponding real HR target, and G(z) the reconstructed image.

$$L_{pixel} = \ mean\ |G(z)\ -\ x|$$

Perceptual and adversarial generator loss

The frozen VGG19 feature extractor compares generated and real images in feature space. Its inputs use VGG19 preprocessing. The resulting features are cast to float32 and divided by 12.75 before calculating mean squared feature error. The same feature preprocessing and divisor are applied to both images.

$$L_{content} = \ mean\ \{[(\varphi(G(z))\ /\ 12.75)\ -\ (\varphi(x)\ /\ 12.75)]^2\}$$

$$L_G = L_{pixel} + \ 0.01\, L_{content} + \ 0.001\ \ BCE(1,\ D(G(z)))$$

L1 penalizes changes in pixel content. The VGG term penalizes differences in pretrained feature activations. The adversarial term encourages G to produce images that D classifies as real. These terms have different numeric scales; their coefficients do not by themselves show their relative contribution. For this run, the weighted VGG term dominated the numerical generator objective.

Discriminator objective

$$L_D = \ 0.5\ [BCE(0.9,\ D(x))\ +\ BCE(0,\ D(G(z)))]$$

Real-image targets are smoothed to 0.9 and generated-image targets are 0.0. The generator's own adversarial target remains 1.0. stop_gradient prevents the discriminator loss from updating G. In the generator-loss discriminator call, training=False prevents that call from updating D's batch-normalization moving statistics.

One adversarial minibatch

1. Load aligned HR targets, apply the shared training flip and construct the corresponding 32 × 32 LR inputs.
2. Generate SR images and evaluate D on real HR, detached SR for its own loss, and SR for the generator objective.
3. Evaluate frozen VGG19 features and calculate pixel, perceptual, generator-adversarial and discriminator losses.
4. Use separate gradient tapes and separate Adam optimizers to update G and D. Both optimizers use learning rate 0.0001.

On the recorded Tesla T4 runtime, mixed_float16 reduces the precision of many internal operations. Losses and final outputs are handled in float32, and each GAN optimizer uses loss scaling. The code checks loss values for nonfinite results. XLA compilation is disabled. This combines memory efficiency with explicit numerical checks; it does not make results independent of software versions, random transformations or GPU kernels.

Interpreting the logged loss magnitudes

At adversarial epoch 150, mean pixel L1 was 0.042345, VGG content MSE was 782.088376, and the unweighted adversarial BCE was 0.325171. Their weighted sum was approximately 7.863554. The content contribution was about 7.820884, much larger than the other two weighted terms. Generator and discriminator losses must therefore not be compared as if they shared a common scale or represented percentages of correct classifications.

7 Training completion and learning behavior

The saved history contains five consecutive warm-up records and 150 consecutive adversarial records. Every record reports 15,705 images_seen. With a batch size of 32, a complete fitting pass contains 491 batches, including its final partial batch. The adversarial stage therefore made 2,355,750 fitting-image presentations across 150 passes. These are repeated presentations of the same fitting subjects, not millions of independent images.

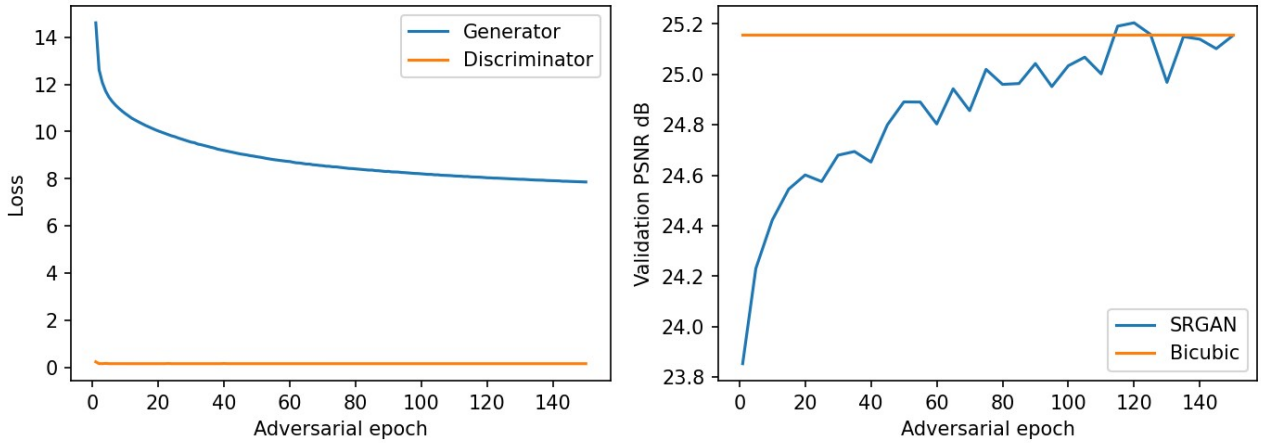


Figure 4. Recorded adversarial generator/discriminator losses and validation PSNR. Bicubic is fixed because the validation images and interpolation method do not change with training.

Training measure	Beginning	End or best
Warm-up pixel L1	0.078459 at epoch 1	0.045968 at epoch 5
Adversarial generator loss	14.609399 at epoch 1	7.863554 at epoch 150
Discriminator loss	0.240983 at epoch 1	0.162547 at epoch 150
Validation SR PSNR	23.853170 dB at epoch 1	25.153383 dB at epoch 150
Best sampled validation SR PSNR	—	25.204354 dB at epoch 120
Validation SR SSIM	0.644611 at epoch 1	0.698782 at epoch 150
Validation bicubic baseline	25.158005 dB	SSIM 0.710686

What the curves support

Generator loss fell while validation PSNR rose by approximately 1.3002 dB from the first adversarial epoch to the last. The curve improved quickly at first and fluctuated near the bicubic PSNR level later. The highest sampled SR PSNR occurred at epoch 120, not epoch 150. The final epoch was nevertheless used to generate B's data, maintaining the predefined complete-training procedure.

The discriminator loss decreased and then became nearly flat. This shows a stable logged objective, but does not establish adversarial equilibrium, prove that all images became realistic, or measure D's accuracy. The validation and test reconstruction metrics, actual examples, and downstream classification results provide separate evidence. No GAN early stopping or test-based generator selection was applied.

8 Qualitative reconstruction examples



Figure 5. Four held-out examples, two cats and two dogs. Columns show LR 32 × 32, bicubic 128 × 128, SRGAN 128 × 128, and the real HR target. LR is displayed enlarged for comparison; its stored resolution remains 32 × 32.

Visual observations

Bicubic enlargement smooths the LR pixels and produces blurred boundaries and fur detail. SRGAN adds stronger local texture and more distinct edges in several examples, especially around fur and facial features. Fine grain or patterned texture is also visible in the generated images. Some of that texture differs from the real target, so sharper appearance cannot be treated as recovery of the original fine detail.

The examples preserve the broad identity of each source animal because the task is conditional reconstruction. They are not independently sampled new cats or dogs. These four images illustrate behavior; the quantitative results below cover all 7,479 test IDs and should carry more weight than a small visual sample.

9 Evaluation design and classifier results

Matched input conditions

The primary comparison evaluates A and B on exactly the same real 128 × 128 test images. Two additional conditions evaluate both models on the same bicubic-enlarged LR images and both on the same SRGAN reconstructions. Holding test identity and input condition constant makes each A/B comparison interpretable. Comparing A on real targets with B on SR images would change both the trained model and the test-image information at once.

Dog is the positive class. Accuracy, binary F1, precision and recall use probability threshold 0.5. ROC AUC uses continuous p_dog values from prediction CSVs, not thresholded labels. AUC describes ranking across thresholds; it does not supply the operating-point error count. All six saved metric rows were reproduced exactly from those CSVs [5].

Model	Test input	Accuracy %	F1	ROC AUC	Precision	Recall
A	real	98.0612	0.980592	0.997993	0.980986	0.980198
B	real	97.1253	0.971774	0.997754	0.953866	0.990367
A	bicubic	87.9262	0.868578	0.962237	0.952138	0.798501
B	bicubic	89.8516	0.900197	0.969225	0.884953	0.915975
A	srgan	94.0500	0.940051	0.986829	0.946555	0.933637
B	srgan	94.3442	0.944393	0.989525	0.928165	0.961199

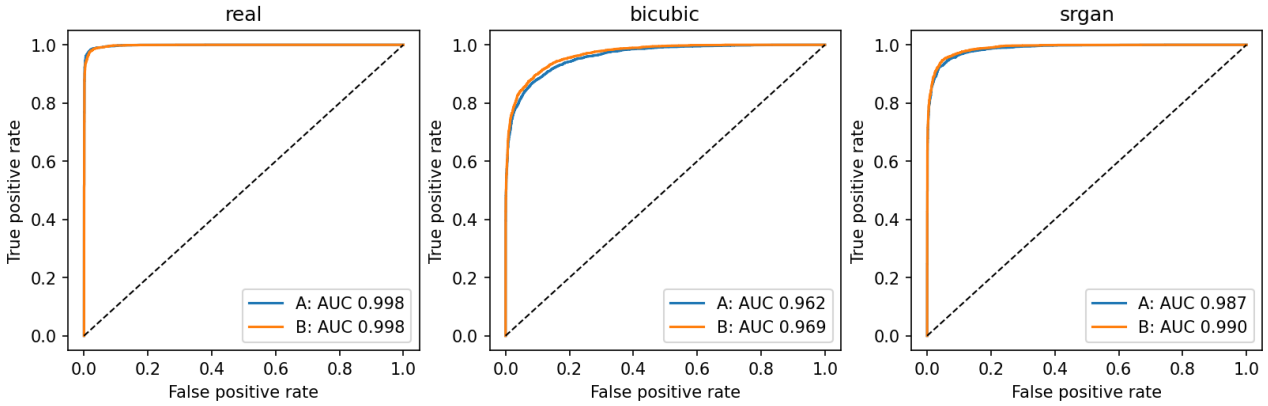


Figure 6. Recorded ROC curves for both classifiers under each matched input condition. The plot legends round AUC to three decimals; the table reports six decimals, revealing the small real-image AUC difference.

Definitions used in the calculations

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}; \quad Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2TP}{2TP + FP + FN}$$

True positives are correctly predicted dogs, true negatives are correctly predicted cats, false positives are cats predicted as dogs, and false negatives are dogs predicted as cats. The binary F1 values in the table are for the dog class; they are not macro-averaged F1 scores. The nearly balanced test set makes accuracy useful, while precision and recall reveal the direction of mistakes.

10 Confusion matrices and worked calculations

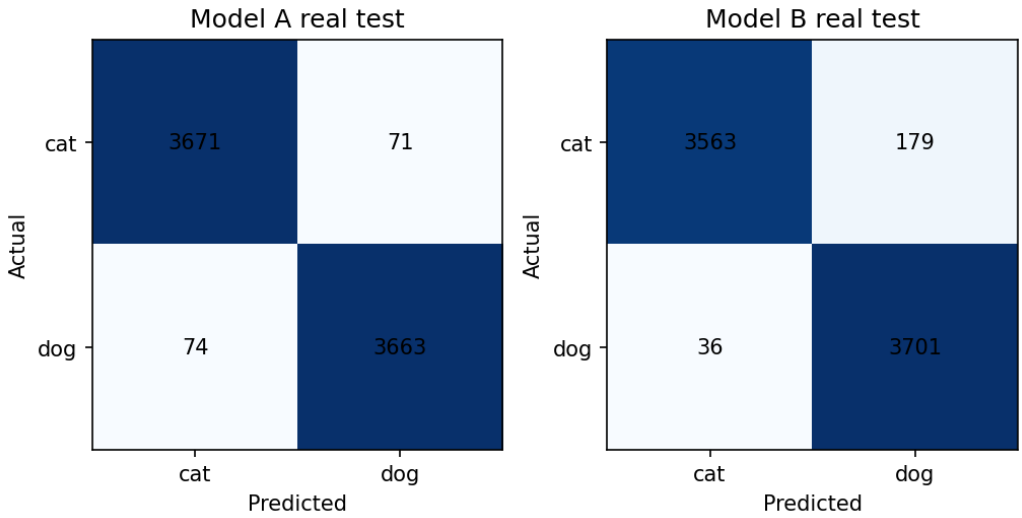


Figure 7. Real-test confusion matrices. Rows are actual classes and columns are predicted classes. Each matrix contains 3,742 actual cats and 3,737 actual dogs.

Classifier	TN	FP	FN	TP	Total errors
A	3,671	71	74	3,663	145
B	3,563	179	36	3,701	215

Classifier A

$$Accuracy = \frac{3671 + 3663}{7479} = \frac{7334}{7479} = 0.980612 = 98.0612\%$$

$$Precision = \frac{3663}{3663 + 71} = 0.980986; \quad Recall = \frac{3663}{3663 + 74} = 0.980198$$

$$F1 = \frac{2 \times 3663}{2 \times 3663 + 71 + 74} = \frac{7326}{7471} = 0.980592$$

Classifier B

$$Accuracy = \frac{3563 + 3701}{7479} = \frac{7264}{7479} = 0.971253 = 97.1253\%$$

$$Precision = \frac{3701}{3701 + 179} = 0.953866; \quad Recall = \frac{3701}{3701 + 36} = 0.990367$$

$$F1 = \frac{2 \times 3701}{2 \times 3701 + 179 + 36} = \frac{7402}{7617} = 0.971774$$

The change in error type

B missed 38 fewer dogs than A, reducing false negatives from 74 to 36. It also mislabeled 108 more cats as dogs, increasing false positives from 71 to 179. The net result was 70 additional errors and a 0.9360 percentage-point reduction in accuracy. B's higher dog recall therefore came with lower precision and lower binary F1 on real inputs.

Both real-input AUCs are near 0.998; these error counts use threshold 0.5. Calibration was not measured. Any future threshold adjustment must use validation data and be assessed once on the held-out test set.

11 Reconstruction fidelity and downstream effects

Test reconstruction fidelity

PSNR and structural similarity compare reconstruction output with the corresponding real HR target. Values are averaged per image across the entire test set, in RGB, with maximum signal value 1 and no border crop. PSNR is capped at 100 dB for exact matches so exports remain finite. The code uses tf.image.psnr and tf.image.ssim. Higher values indicate better agreement under these measures, but do not certify that a generated texture is correct.

$$PSNR = 10 \log_{10}\left(\frac{1}{MSE}\right) dB$$

Test reconstruction	Mean PSNR dB	Mean SSIM
Bicubic	25.063898	0.707662
SRGAN	25.037427	0.694663
SRGAN minus bicubic	−0.026472	−0.012999

SRGAN did not outperform bicubic on these aggregate fidelity measures. Its test PSNR was very close to the baseline, while SSIM was lower by approximately 0.0130. This agrees with the visual possibility of sharper-looking but altered texture. A perceptual/adversarial objective need not maximize pixel-wise reconstruction fidelity [2].

Classification effects under the same input condition

Comparison	Accuracy change	Change in correct predictions
B minus A on real targets	−0.9360 percentage points	70 fewer correct
B minus A on bicubic inputs	+1.9254 percentage points	144 more correct
B minus A on SRGAN inputs	+0.2942 percentage points	22 more correct
SRGAN minus bicubic inputs for A	+6.1238 percentage points	458 more correct
SRGAN minus bicubic inputs for B	+4.4926 percentage points	336 more correct

The lower PSNR/SSIM values did not prevent SRGAN inputs from improving classification relative to bicubic inputs in both classifiers. Conversely, training B on generated images did not improve accuracy on real targets. These results answer different questions: reconstruction can change task-relevant features without improving average pixel agreement, and adaptation to generated inputs need not transfer as well to real inputs.

The reconstruction-quality routine evaluates floating-point G outputs. Classifier SR inputs come from the rounded uint8 reconstruction cache, whereas bicubic classifier inputs remain floating point. This small pipeline distinction should be preserved for exact reproduction and controlled in a future ablation. The results above describe the implemented pipelines, rather than proving that one image-quality metric directly causes classification performance.

12 Checkpoints software environment and recorded time

Full state saved after each completed epoch

The save interval was n = 1. GAN checkpoints track G, D, both Adam optimizers, loss-scaling state, warm-up epoch and adversarial epoch. CheckpointManager retains the three most recent GAN checkpoints. Classifier head and fine-tuning stages each maintain a latest checkpoint and a best-validation checkpoint, including model, optimizer, completed epoch and best loss [4]. Final .keras files are separately exported for inference.

After a disconnect, reopening the same experiment reconstructs the code objects and restores their tracked state. Completed classifiers are loaded from their exported files; prepared and generated image caches are reused when their identities match. Restarting with a different experiment name points to a different output folder. The original folder, configuration, split and checkpoint companion files must remain intact. Work since the latest completed saved epoch can be lost in an abrupt interruption.

Recorded component	Version or setting
Python	3.13.16
TensorFlow	2.21.0
Keras	3.13.2
GPU	Tesla T4
Numeric policy	mixed_float16
NumPy	2.1.3
Pillow	11.3.0
scikit-learn	1.6.1
Matplotlib	3.10.0
ReportLab	5.1.0

Durations recorded by the implementation

Stage	Logged seconds	Logged minutes
Generator warm up	260.48	4.34
150 adversarial epochs	18,906.36	315.11
Classifier A head and fine tuning	152.55	2.54
Classifier B head and fine tuning	145.95	2.43
Sum of recorded intervals	19,465.34	324.42

The summed recorded intervals equal approximately 5 hours 24 minutes 25 seconds. They are not a measurement of end-to-end wall-clock time. The GAN epoch timer ends before post-epoch validation, checkpoint writes and preview plots; classifier intervals include their fit, validation and checkpoint operations. Dataset preparation, generating the reconstruction cache, final testing, exports and time between sessions add further elapsed time.

The notebook checks a ten-hour GAN session budget at epoch boundaries, but resource availability can terminate a session sooner. Precision must match the saved experiment: a run saved with mixed precision requires the same optimizer structure when restoring. The updated split cell reads saved settings and IDs before continuing. If no GPU is detected, it pauses that mixed-precision resume path instead of silently altering the experiment.

13 Procedure to reproduce the experiment

Preparation and data access

1. Download the repository notebook and this report. Obtain the same Assignment 1 image archive, or reconstruct its prepared 128 × 128 images from the supplied RGB cache and manifest using Appendix A.
2. Open the notebook in Google Colab and choose a GPU runtime. Run Section 1 to mount the intended Drive account and confirm that TensorFlow lists a GPU.
3. In Section 2 set DATA_SOURCE to the real archive or image-folder path. Set DRIVE_BASE to a writable output folder and use cats_dogs_srgan_midterm_v1 when resuming the existing run. Use a new experiment name for an independent reproduction.
4. Use the recorded configuration below. Keep TensorFlow and Keras compatible with the saved checkpoint environment when resuming; the recorded package versions are listed in Section 12.
5. Run image preparation. Confirm 24,930 prepared RGB images, the expected label counts and the dataset fingerprint. Review exclusion and duplicate logs. The provided fingerprint identifies ordered prepared pixels and labels, not a ZIP filename or modification time.

```
seed = 42
hr_size = 128; lr_size = 32
gan_batch = 32; classifier_batch = 32
warmup_epochs = 5; gan_epochs = 150
head_epochs = 5; fine_epochs = 3
g_filters = 32; g_blocks = 8; d_filters = 32
vgg_layer = "block3_conv4"
pixel_weight = 1.0
perceptual_weight = 0.01; adversarial_weight = 0.001
checkpoint_every = 1; validate_every = 5
session_hours = 10.0; mixed_precision = True
imagenet_weights = "imagenet"
```

Training evaluation and export

6. Load the saved split or create the deterministic 70/30 outer split and inner validation split for a new run. Confirm 15,705 fitting, 1,746 validation and 7,479 testing IDs. Demonstrate the transformations and scaled inputs.
7. Train A for five head epochs and three fine-tuning epochs. Construct G, D and frozen VGG19. Complete all five warm-up epochs and all 150 adversarial epochs, visiting the full fitting set in every pass.
8. Freeze G for generation. Cache its final reconstructions with unchanged source labels and split IDs. Initialize B from classifier_initial.keras and train with the same classifier schedule, using generated fitting and validation inputs.
9. Evaluate both classifiers under real, bicubic and SRGAN test conditions. Keep threshold 0.5. Calculate ROC AUC from p_dog and save predictions, not just the six summary rows. Compute SRGAN and bicubic PSNR/SSIM against real targets.
10. Save the executed notebook with visible outputs. Export the metric CSV/JSON files, history files, figures, final models or accessible model links, and this report. Recheck the expected results in Sections 9–11; repeated GPU training can vary numerically even with the same seed.

For an interrupted run, repeat initialization and data-loading cells in order with the same experiment folder. Look for the restored epoch counters before allowing training to continue. Do not overwrite experiment_spec.json to bypass a mismatch; restore the compatible configuration or resolve a changed dataset first.

14 Discussion limitations and conclusions

What the experiment demonstrated

The required fourfold super-resolution pipeline was trained and evaluated successfully. The generator completed 150 adversarial epochs after reconstruction warm-up, B was trained on generated fitting images, and both classifiers were evaluated on the same held-out identities under each input condition. The records support completion of the computational requirements and a reproducible comparison, rather than a general claim that generated images always improve a classifier.

On real targets, A was the stronger classifier at the fixed operating threshold. B's recall increased, but the increase in cats incorrectly called dogs outweighed the reduction in missed dogs. Under degraded inputs, B adapted better: it gained 1.9254 percentage points over A on bicubic images and 0.2942 percentage points on SRGAN images. The latter is only 22 additional correct predictions among 7,479 test images.

Paired predictions rather than only aggregate percentages

Input condition	Only A correct	Only B correct	Both wrong
Real	109	39	106
Bicubic	301	445	458
SRGAN	121	143	302

These counts were derived by matching the prediction files by source-image ID. They show that the models did not make exactly the same mistakes, even when aggregate AUCs were similar. For SRGAN inputs, B corrected 143 cases that A missed, while A corrected 121 cases that B missed. The net difference is 22. No repeated-seed confidence intervals or formal significance claims are reported.

Limits of the evidence

- One dataset, split and training seed were evaluated. Results do not establish the average gain across independent training runs or unseen image sources.
- The 70/30 split is image-level. Exact duplicates were checked, but related images of the same animal or source collection may remain across subsets.
- LR inputs were created by a known bicubic degradation. Real small photographs may contain different blur, noise, compression, occlusion or lighting.
- B sees generated versions of existing labeled subjects. This is a change in training representation, not an increase in independent labeled examples.
- The model is a compact SRGAN adaptation with a feature-dominated composite loss. No systematic ablation of loss weights, generator width, feature layer or mixed real/SR training was performed.
- PSNR and SSIM were slightly worse than bicubic on the test set, and the examples show generated texture. Visual sharpness should not be described as guaranteed recovery of true detail.

Conclusion and useful next experiments

For this run, retain A when real 128 × 128 images are available. B is a better matched classifier for the reconstructed input conditions, although its SRGAN-input advantage over A is small. SRGAN preprocessing improved classification relative to bicubic inputs for both classifiers, while not improving the measured reconstruction fidelity. Future work could repeat several seeds, group related images before splitting, test real low-resolution photographs, standardize quantization between interpolation methods, and compare mixed real/SR training and validation-calibrated thresholds. Those experiments were not part of the reported results.

15 GitHub contents references and data availability

Repository contents

The repository should contain the executed notebook, the reproducibility documentation and the exported evidence needed to verify the experiment. The small results ZIP is an export container; its contents should be visible in the repository where practical. Large RGB caches and model binaries can be provided through accessible download links. Keep training checkpoints available for resumption, and keep each TensorFlow checkpoint prefix together with its data and index files.

File or folder	Purpose
SRGAN_Midterm_Colab.ipynb	Executed implementation with scaling, transformations and final outputs
README.md	Entry point with run instructions, data/model links and measured summary
SRGAN_Detailed_Report.docx or .pdf	Detailed methods, calculations, figures and interpretation
experiment_spec.json	Fixed architecture, hyperparameters, precision and data fingerprint
split_indices.json and image_manifest.json	Exact memberships, labels, source paths and prepared-image hashes
results/classifier_metrics.csv and .json	Six matched-input A/B metric rows
results/predictions_*.csv	Per-image IDs, true labels, probabilities and hard predictions
results/*history.json and environment.json	Epoch records, completed budgets, durations and package versions
results/*.png	Scaled, transformed, reconstructed, ROC, confusion and learning figures
models/ or download links	Final G, D, A and B; initialization and selection records when available

Submit only the GitHub repository URL under Canvas Assignment/Mid-Term Exam. The report belongs inside that repository. Verify that the evaluator can open the linked dataset and model assets; a working link in the author's signed-in session does not establish access for another reader.

Project records and available assets

[1] Recorded experiment folder with data caches metadata models and results

Verified source export SRGAN_GitHub_Results.zip

Prepared RGB image cache dataset_128_rgb.npz

Technical references

[2] C Ledig et al Photo Realistic Single Image Super Resolution Using a Generative Adversarial Network CVPR 2017

[3] Keras Applications MobileNet models and preprocessing

[4] TensorFlow Training checkpoints

[5] scikit learn ROC AUC score documentation

[6] Keras Applications VGG16 and VGG19 preprocessing

Appendix A Exact data verification and cache reconstruction

The prepared data fingerprint is the SHA-256 of the ordered prepared-pixel hashes followed by their numeric labels. It identifies the exact 128 × 128 RGB inputs used by this experiment. ZIP paths and file modification times belong to a separate source signature; changing storage locations can change that signature even if the prepared pixels are identical.

```
01a704750e08bc899b4388540ea60e9d7fd34b5fcf9336fc5faf4158b298da67
```

If the Assignment 1 archive is unavailable, the provided RGB cache and image manifest preserve the actual prepared targets and labels. Download dataset_128_rgb.npz, image_manifest.json and dataset_cache_metadata.json from the project folder. The following CPU-only preparation step recreates lossless PNG images in the manifest's class-folder structure. It does not train a model or reproduce higher-resolution originals that were already downscaled.

```
import json, hashlib
from pathlib import Path
import numpy as np
from PIL import Image

CACHE = Path("/content/dataset_128_rgb.npz")
MANIFEST = Path("/content/image_manifest.json")
META = Path("/content/dataset_cache_metadata.json")
DEST = Path("/content/rebuilt_srgan_dataset")

records = json.loads(MANIFEST.read_text())
meta = json.loads(META.read_text())
with np.load(CACHE, allow_pickle=False) as data:
    images, labels = data["images"], data["labels"]
assert len(images) == len(records) == meta["count"]
ordered = []
for record in records:
    i = record["index"]
    digest = hashlib.sha256(images[i].tobytes()).hexdigest()
    assert digest == record["sha256"]
    assert int(labels[i]) == record["label"]
    ordered.append(digest + str(record["label"]))
assert hashlib.sha256("".join(ordered).encode()).hexdigest() == meta["fingerprint"]

DEST.mkdir(parents=True, exist_ok=True)
assert not any(DEST.iterdir()), "Use a new empty output folder."
for record in records:
    target = (DEST / record["source"]).with_suffix(".png").resolve()
    assert target.is_relative_to(DEST.resolve())
    target.parent.mkdir(parents=True, exist_ok=True)
    Image.fromarray(images[record["index"]]).save(target)
print("Prepared data verified and reconstructed:", DEST)
```

Set DATA_SOURCE to /content/rebuilt_srgan_dataset for a fresh reproduction. Load the supplied split IDs for identical membership, and verify the resulting dataset fingerprint before training. PNG encoding preserves the cached RGB values; folder/extension changes are packaging changes. Rebuilding from the original raw-image archive remains the way to reproduce its earlier RGB conversion and resizing step.

The independent report verification also recomputed accuracy, binary F1, ROC AUC, precision and recall from each saved prediction CSV. It checked ID ordering against split_indices.json, labels against the manifest, and predictions against p_dog ≥ 0.5. All six rows matched the saved metric JSON to numerical precision. The history check found all five warm-up epochs and all 150 adversarial epochs, with the full fitting-image count in every record.